

Algorytmy Ewolucyjne – Projekt 2: Problem Plecakowy

Algorytm genetyczny z kodowaniem binarnym dla problemu 0/1 plecakowego

Mikołaj Wróbel 337021

32 przedmioty, $W = 30\%$ sumy wag, selekcja turniejowa, krzyżowanie dwupunktowe, mutacja bitowa.

```
In [47]: %load_ext autoreload
%autoreload 2

import sys, os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

sys.path.insert(0, os.path.join(os.getcwd(), 'find_best_backpack'))

from genetic_algorythym.genetic_algorythym import genetic_evolution, chromosome
from genetic_algorythym.genetic_algorythym_parameters import return_genetic_algorith

# --- Wczytanie przedmiotów ---
items_df = pd.read_csv('get_all_items/items_337021.csv', sep=';')
weights = items_df['Waga'].values.astype(float)
values = items_df['Wartosc'].values.astype(float)

total_weight = weights.sum()
MAX_WEIGHT = 0.30 * total_weight

print(f"Suma wag wszystkich przedmiotów : {total_weight:.1f}")
print(f"Maksymalna waga plecaka W=30% : {MAX_WEIGHT:.2f}")

# --- Ustawienie zmiennych klasowych chromosomu ---
chromosome.weights = weights
chromosome.values = values
chromosome.max_weight = MAX_WEIGHT
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

Suma wag wszystkich przedmiotów : 14.4

Maksymalna waga plecaka W=30% : 4.32

Lista przedmiotów

```
In [48]: from IPython.display import HTML, display
display(items_df.style.format({'Waga': '{:.1f}', 'Wartosc': '{:.0f}'}))
```

	ID_Przedmiotu	Waga	Wartosc
0	1	0.6	63
1	2	0.2	39
2	3	0.6	72
3	4	0.9	1
4	5	0.4	78
5	6	0.2	90
6	7	0.7	95
7	8	0.2	7
8	9	0.7	58
9	10	0.7	62
10	11	0.4	46
11	12	0.2	9
12	13	0.6	70
13	14	1.0	14
14	15	0.4	4
15	16	0.2	36
16	17	0.7	63
17	18	0.3	62
18	19	0.3	41
19	20	0.2	50
20	21	0.6	32
21	22	0.3	56
22	23	0.9	85
23	24	0.2	66
24	25	0.1	98
25	26	0.3	18
26	27	0.5	85
27	28	0.2	18
28	29	0.3	10
29	30	0.8	88
30	31	0.2	30
31	32	0.5	33

Parametry algorytmu

W celu doboru optymalnego rozmiaru turnieju testowane są cztery wartości $k \in \{1, 2, 5, 10\}$ przy stałych pozostałych parametrach.

Parametr	Wartość
Liczność populacji N	100
Potomkowie elitarni N_e	5
Potomkowie skrzyżowani N_c	80
Potomkowie zmutowani N_m	15
Prawdopodobieństwo mutacji p_m	$1/32 \approx 0,031$
Prawdopodobieństwo krzyżowania p_c	0,8
Liczba pokoleń	300
Warunek zatrzymania	stała liczba pokoleń (300)

```
In [49]: base_params = return_genetic_algorithm_parameters()

tournament_sizes = [1, 2, 5, 10]
results = {}

for k in tournament_sizes:
    print(f"Uruchamianie GA (turniej k={k})...")
    params = {**base_params, "tournament_size": k}
    ga = genetic_evolution(**params)
    best_chrom, best_fitness, history = ga.genetic_algorithm()
    results[k] = {"history": history, "best": best_chrom, "fitness": best_fitness}
    print(f" Najlepsza wartość: {best_fitness:.0f}\n")
```

```
Uruchamianie GA (turniej k=1)...
Najlepsza wartość: 868
```

```
Uruchamianie GA (turniej k=2)...
Najlepsza wartość: 875
```

```
Uruchamianie GA (turniej k=5)...
Najlepsza wartość: 873
```

```
Uruchamianie GA (turniej k=10)...
Najlepsza wartość: 875
```

Wykresy wartości funkcji celu

Dla każdego uruchomienia: minimum, średnia, maksimum (lewa oś) oraz wariancja (prawa oś) w funkcji numeru pokolenia.

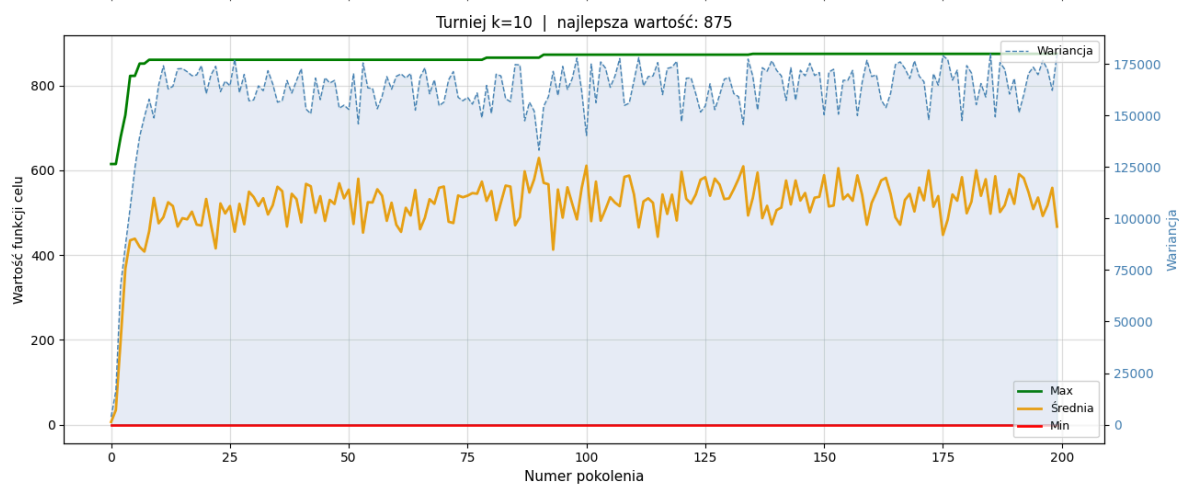
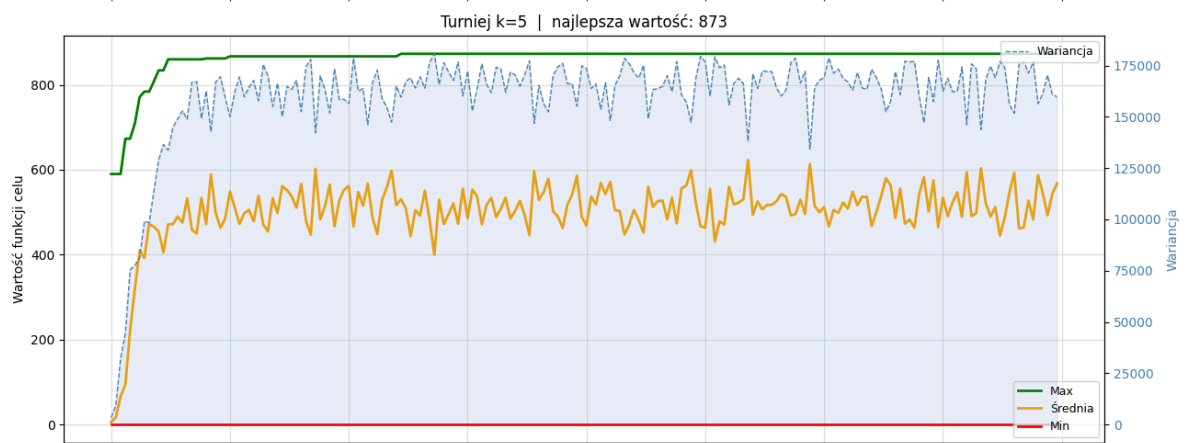
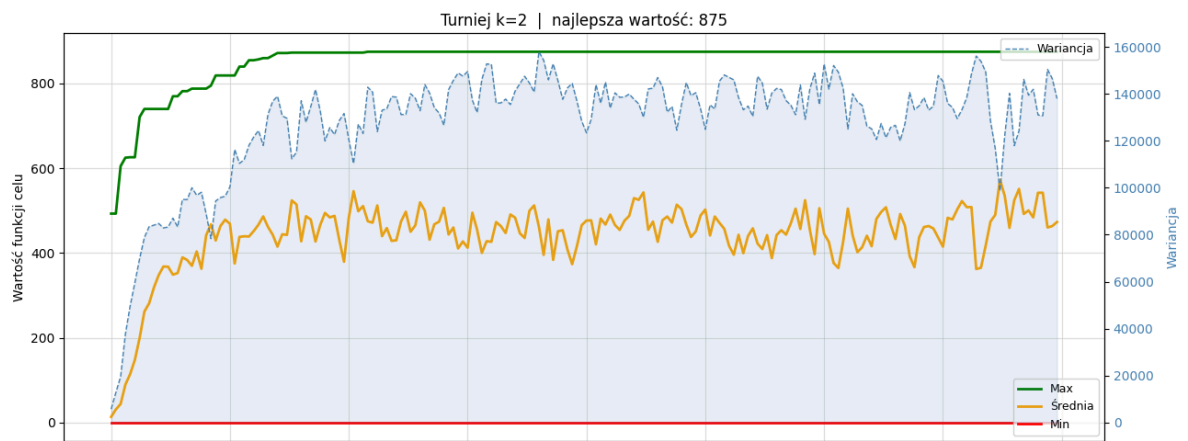
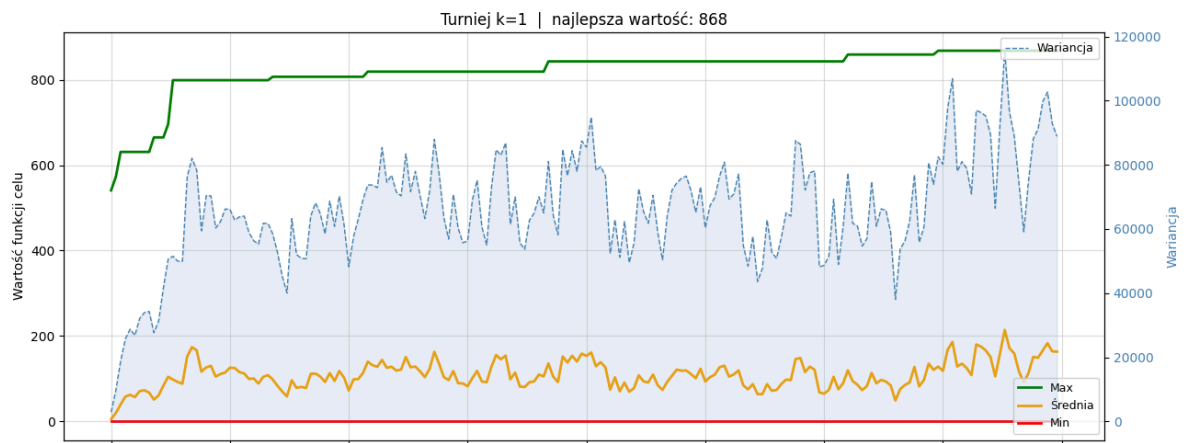
```
In [50]: fig, axes = plt.subplots(len(tournament_sizes), 1, figsize=(13, 5 * len(tournament_

for ax, k in zip(axes, tournament_sizes):
    h = results[k]["history"]
    gens = range(len(h))

    ax.plot(gens, [e["max"] for e in h], label="Max", color="green", linewidth=2)
    ax.plot(gens, [e["mean"] for e in h], label="Średnia", color="orange", linewidth=2)
    ax.plot(gens, [e["min"] for e in h], label="Min", color="red", linewidth=2)
    ax.set_ylabel("Wartość funkcji celu", fontsize=10)
    ax.set_title(f"Turniej k={k} | najlepsza wartość: {results[k]['fitness']:.0f}")
    ax.legend(loc="lower right", fontsize=9)
    ax.grid(True, alpha=0.4)

    ax2 = ax.twinx()
    ax2.fill_between(gens, [e["var"] for e in h], alpha=0.12, color="steelblue")
    ax2.plot(gens, [e["var"] for e in h], color="steelblue", linewidth=1, linestyle='--')
    ax2.set_ylabel("Wariancja", color="steelblue", fontsize=10)
    ax2.tick_params(axis="y", labelcolor="steelblue")
    ax2.legend(loc="upper right", fontsize=9)

axes[-1].set_xlabel("Numer pokolenia", fontsize=11)
plt.tight_layout()
plt.show()
```



Dobór optymalnych parametrów

Warunek zatrzymania: stała liczba pokoleń (300). Dobrana empirycznie — na podstawie powyższych wykresów widać, że wartość najlepszego osobnika stabilizuje się przed pokoleniem 250 we wszystkich testowanych konfiguracjach. Dalsze iteracje nie poprawiają wyniku.

Metoda selekcji i rozmiar turnieju k : jako kryterium doboru przyjęto najwyższą wartość funkcji celu uzyskaną przez najlepszego osobnika w całym przebiegu. Poniższa tabela podsumowuje wyniki wszystkich uruchomień — konfiguracją optymalną jest ta z najwyższą wartością.

```
In [51]: summary_rows = []
for k in tournament_sizes:
    h = results[k]["history"]
    summary_rows.append({
        "Turniej k": k,
        "Najlepsza wartość": results[k]["fitness"],
        "Średnia końcowa": h[-1]["mean"],
        "Wariancja końcowa": h[-1]["var"],
    })

summary_df = pd.DataFrame(summary_rows).set_index("Turniej k")
optimal_k = max(tournament_sizes, key=lambda k: results[k]["fitness"])

display(
    summary_df.style
        .format({"Najlepsza wartość": "{:.0f}", "Średnia końcowa": "{:.1f}", "Wariancja
        .highlight_max(subset=["Najlepsza wartość"], color="#c6efce")
    )
    print(f"\nOptymalna konfiguracja: turniej k={optimal_k} → wartość: {results[optimal_k]}
```

	Najlepsza wartość	Średnia końcowa	Wariancja końcowa
Turniej k			
1	868	163.5	88832.5
2	875	473.2	137995.9
5	873	567.7	159799.2
10	875	467.2	179734.0

Optymalna konfiguracja: turniej k=2 → wartość: 875

Najlepsze rozwiązanie

```
In [52]: genes = results[optimal_k]["best"].genes
sel_mask = np.array(genes, dtype=bool)

total_sel_weight = float(np.dot(genes, weights))
total_sel_value = float(np.dot(genes, values))

print(f"Optymalna konfiguracja: turniej k={optimal_k}\n")
```

```

print("Wektor binarny rozwiązania:")
print(genes)

print(f"\nWybrane przedmioty ({sel_mask.sum()} z {len(genes)}):")
display(items_df[sel_mask].style.format({"Waga": "{:.1f}", "Wartosc": "{:.0f}"}))

print(f"Sumaryczna waga   : {total_sel_weight:.1f} ≤ {MAX_WEIGHT:.2f} (limit W)")
print(f"Sumaryczna wartość: {total_sel_value:.0f}")

```

Optymalna konfiguracja: turniej k=2

Wektor binarny rozwiązania:

[0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0]

Wybrane przedmioty (14 z 32):

	ID_Przedmiotu	Waga	Wartosc
1	2	0.2	39
2	3	0.6	72
4	5	0.4	78
5	6	0.2	90
6	7	0.7	95
15	16	0.2	36
17	18	0.3	62
19	20	0.2	50
21	22	0.3	56
23	24	0.2	66
24	25	0.1	98
26	27	0.5	85
27	28	0.2	18
30	31	0.2	30

Sumaryczna waga : 4.3 ≤ 4.32 (limit W)

Sumaryczna wartość: 875